# Facebook Friend Recommendation using Graph Mining

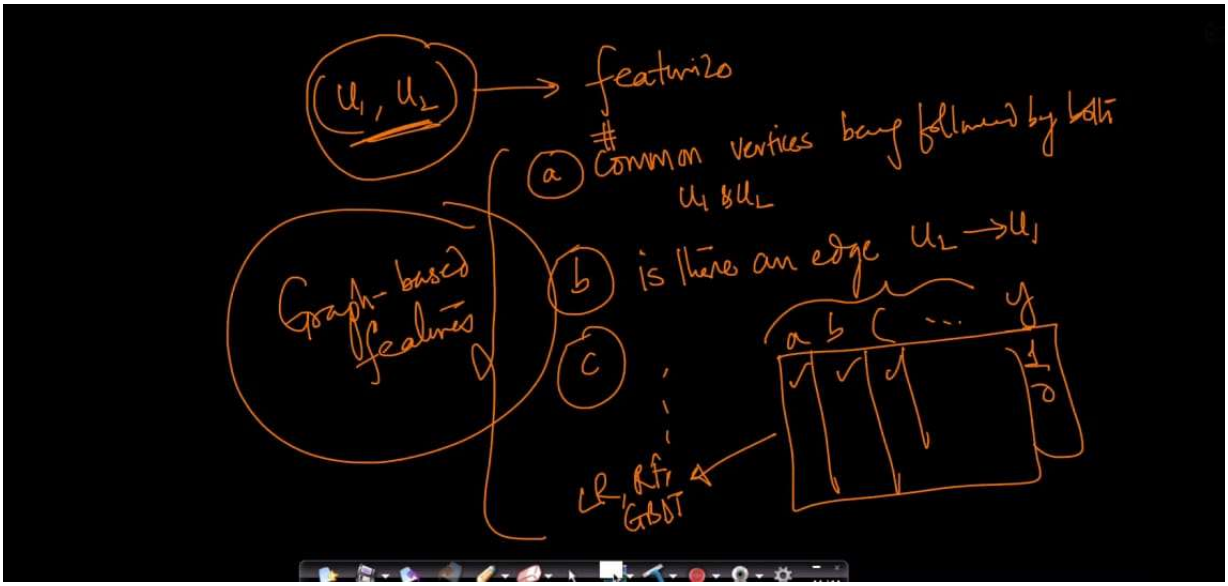
1) Understanding of the problem
2) https://swetapadma449.medium.com/facebook-friend-recommendation-using-graph-mining-8d8d62153d14

1) Given a directed social graph, have to predict missing links to recommend users (Link Prediction in graph).

2) How are graphs be used in future to recommend friends. A proper graphical / link based problem statement.

3) Pure graph -link based problem. We got to know about the dataset which has only 2 columns preserving the user information. The columns represent user as a source (follower) and destination (following). There is million of data. Since we only have two columns and those are also link based we will have to work on some good feature engineering. The data is dynamic. Suppose today I followed my school friends and tomorrow I will follow my new friend and all. If they would have given us time as a constraint then it would have been great since time is a good constraint to know the user when follows when and also get many insights.

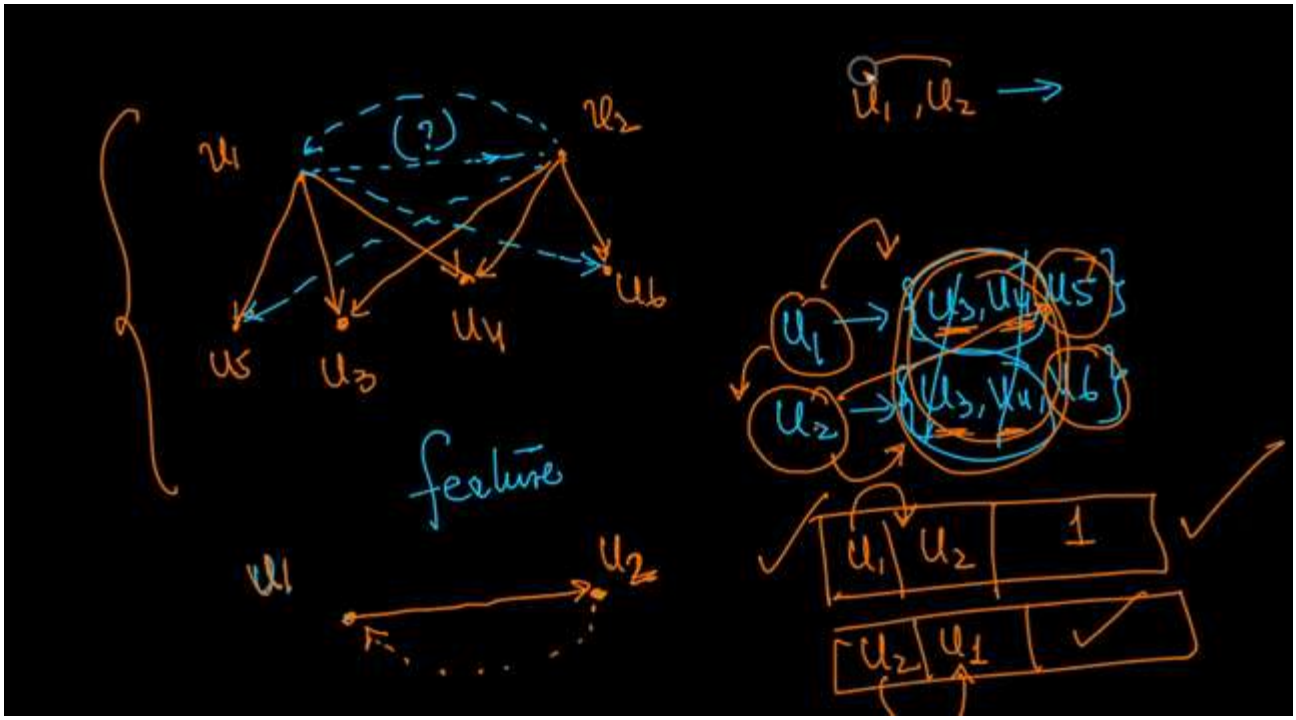
4) Just now refereed the dataset. There are many of the cases such that the follower and the following are vice-versa like if tu mujhe follow karta hai then mai tujhe follow karta hu but still we will have to consider a case where if u follow me it is not necessary that I will follow u and hence we can recommend it.

5) 


6) Refer the above images in order to get a hint like such are some of the feature engineering techniques that we will have to work with.

7) **test.csv** contains a list of nodes in the social graph. For each node, recommend up to 10 other nodes for that user to follow.
   Submit a 2-column csv file. The first column (source_node) should contain an integer representing the nodes in test.csv, in the same order that they appeared in test.csv. The second column (destination_nodes) should contain a space-delimited series of between 0 and 10 integers, representing the 10 recommended nodes.

8)




9) **Business Objectives & Constraints**

**No low-latency requirement** - Can afford complex computations

**Probability scores are valuable** - Higher probability links should be recommended first

**Performance Metric**: F1-score (balance between precision and recall)

10) **F1-Score**

**Main metric**: F1-Score is the primary evaluation metric

**Rationale**: Both precision and recall are equally important for friend recommendation

**Precision**: Ensures recommended friends are relevant (avoid false positives)

**Recall**: Ensures we don't miss potential good friend connections (avoid false negatives)

11) **F1-Score**: Harmonic mean of precision and recall = 2 × (Precision × Recall) / (Precision + Recall)

**Why F1-Score is Ideal Here**

**Balanced Classes**: Friend recommendation needs balance between precision and recall

**Business Value**:

High precision → Users get relevant friend suggestions

High recall → Don't miss potential meaningful connections

**No Specific Preference**: Neither false positives nor false negatives are significantly worse than the other

**High precision and High recall = High F1**-Score.

**High F1-Score = Happy Users:**

- Users trust the recommendations (high precision)
- Users discover meaningful connections (high recall)
- Platform engagement increases
- Network effects strengthen

12)

**2. Real-World Trade-offs**

**If we optimize only for Precision:**

```
# Conservative model - only recommends when very confident
Precision: 95% ✅ (great recommendations)
Recall: 30% ❌ (misses many good friends)
Result: Users get few but excellent recommendations
```

**If we optimize only for Recall:**

```
# Aggressive model - recommends liberally
Precision: 40% ❌ (many irrelevant suggestions)
Recall: 90% ✅ (catches most potential friends)
Result: Users overwhelmed with poor recommendations
```

**F1-Score finds the sweet spot:**

```
# Balanced model
Precision: 85% ✅
Recall: 85% ✅
F1-Score: 85% ✅
Result: Good mix of quality and quantity
```

13) In original companies like facebook, Instagram and others they themselves updates the recommentaion every night or every week or every 2 days.
So there is no low-latency requirement.

14) In total, we have 1862220(1.86 million) vertices/nodes and 9437520( 9.43 million) edges/links in our directed graph.

We could have metadata like about the city in which the person lives in or in which college they studied or did they study in the same college or some additional information. But, as far as this task is concerned, we have directed graph nodes and edges and have to work on it.
So, this is a purely graph-based link prediction problem.
But, as the network grows, people are following new people. The network is very dynamic in real-world because today I may have discovered my old friend on Facebook and started following them. So as far as the problem is concerned, Facebook gave us a snapshot of the graph at one time. So, there are some constraints as we cannot understand the evolution of the graph.

15)

# 1) FB_EDA-1.ipynb

1) Here we will do basic to advance EDA { sir EDA including and our also }

2) We will generate many of the csv files in future for some or the other use.

3) Here we will make use of a python library named Networkx. In order to work with graph related problems and have inbuilt functions to deal with graph related problems, we use Networkx.
   **What is NetworkX?**
   NetworkX is a **Python library specifically designed for working with networks/graphs**. Think of it as a specialized tool for analyzing relationships between things.
   **Real-World Examples:**

- **Social Networks:** Facebook friendships, Twitter followers
- **Transportation:** Roads connecting cities, flight routes
- **Internet:** Websites linking to each other
- **Biology:** How proteins interact with each other

4) We made a csv file named train_woheader.csv which is same as train.csv but without header. NetworkX's read_edgelist() function expects a **simple text file** with just the data. **In Simple Terms:** NetworkX reads every line as a connection between two people. If we keep headers, it thinks "source_node" and "destination_node" are actual people's names!

5) Now we made a graph of the first 50 samples and removed the number of nodes and edges. Of the first 50 samples the number of unique people or nodes are 66 and there are 50 edges that is connection between the nodes. The average connection of each people in the first 50 is 1.

Matlab har koi kisi ek jaan ko follow karta hai.
It is a directed graph. That is if A follows B then it is not compulsory that B will follow A. Our work at this stage will only be to recommend B to follow A.

6) We have a total of 1862220 people and 9437519 edges through which we will do all the EDA and training .

7) No. of followers of each person
Indegree – means followers
Here we are doing the analysis of no. followers each person is following in order to get our target audience like in our data set do we have people with only high followers {influnecer, founder, singer etc} or we have middle-class or people with low followers { normal people }.

📊 **Three Distinct Zones:**

🟤 **Zone 1: The Flat Area (0 - 1.5M people)**

- **Y-axis:** 0-20 followers
- **What it means: Most people have very few followers**
- **Real world:** Regular users, new accounts, inactive users

🟤 **Zone 2: The Gradual Rise (1.5M - 1.7M people)**

- **Y-axis:** 20-100 followers
- **What it means: Moderately active users**
- **Real world:** People with some social presence

🟤 **Zone 3: The Sharp Spike (1.7M+ people)**

- **Y-axis:** 100-500+ followers
- **What it means: Influencers and popular accounts**
- **Real world:** Content creators, brands, celebrities

---

🧠 **Social Network Science Behind This:**

**The 80/20 Rule in Action:**

- **80% of people:** Have very few followers (0-20)
- **20% of people:** Have most of the follower attention
- **1% of people:** Are super-influencers with hundreds of followers

~1,500,000 people: "I have 0-10 followers" (Regular users)
~200,000 people: "I have 10-50 followers" (Active users)
~100,000 people: "I have 50+ followers" (Popular users)
~Few thousand: "I have 200+ followers" (Influencers)

**"Most users are isolated islands - our job is to build bridges between them!"**

Now I ran and saw the box-plot of our followers. We know that 90 percent of people are have folowers only 0-40 and remaining 10 percent are 50-500 or more.

- **Statistical outliers**: People with 100-500+ followers appear as outliers in the boxplot
- **Business reality**: These are influencers, celebrities, public figures, or popular users
- In traditional data cleaning, we might remove outliers, but here they're **gold!**

**Why We CANNOT Remove Them:**

**a) They Drive the Recommendation Engine:**

- High-follower users are **connectors** in the network
- They bridge different communities
- Removing them would break many potential friendship paths

**b) They Have Different Behavior Patterns:**

- Influencers follow different people than regular users
- They create **hub-and-spoke** patterns in the network
- These patterns are crucial features for ML models

**c) They're Revenue Generators:**

- In real Facebook, these users generate most engagement
- Friend recommendations involving them have higher success rates
- They're more likely to accept/send friend requests

Here after referring to this box-plot we can make a feature that is popular, mid , low and whenever any person create their fb account could follow the popular one and accordingly could get the recommendation of the person he might know and are following that popular one's. This is exactly how platforms like Facebook, Twitter, and LinkedIn bootstrap new user connections.

90 percent of people have follower 40 only.

8) No. of people following

After referring all the box plots and graphs the number of people following is similar to number of followers.
Here also 90 percent people are following less than 40 people.
Out-degree means following – source node.

There are 14 percent people that is 2 lakh people who are only followed by people and they do not follow any one.
And there are 1 lakh people that 10 percent people who has zero followers.
Here also i was thinking for two features - Have followers and Have following. such that if any person has zero following, they will be given qualitative and quantitative recommendation and the recommendation to these people will be strong like will have to find some strong linking of people which are closely related to that person. For zero followers we can recommend all the followers of that person as their first recommendation so that those people could be given high priority rather then others.

- **~14% (200k people)**: Have followers but follow nobody → "**Celebrities/Influencers**"
- **~10% (100k people)**: Have zero followers → "**New/Isolated users**"

**Your Feature Engineering Strategy is EXCELLENT:**

**1. Binary Features:**

- has_followers (0/1)
- has_following (0/1)

**2. Recommendation Strategies by User Type:**

**Type A: Zero Following (200k users - 14%)**

- These are **content consumers/lurkers**
- **Strategy**: Strong qualitative recommendations
- **Method**: Use similarity metrics (Jaccard, Adamic-Adar) to find users with similar interests
- **Priority**: High-confidence recommendations only

**Type B: Zero Followers (100k users - 10%)**

- These are **new/isolated users**
- **Strategy**: Bootstrap their network quickly
- **Method**: Recommend followers of people they follow (2nd degree connections)
- **Logic**: If A follows B, and C follows B, then recommend C to A

Zero Following Users → Need discovery help → Qualitative recommendations

Zero Followers Users → Need visibility boost → Quantitative recommendations

- **Different recommendation algorithms** for each user type
- **Different confidence thresholds** (higher for zero-following users)
- **Different recommendation volumes** (more for zero-followers users)

9) Number of followers and following

Min of no of followers + following is 1

334291 persons having minimum no of followers + following

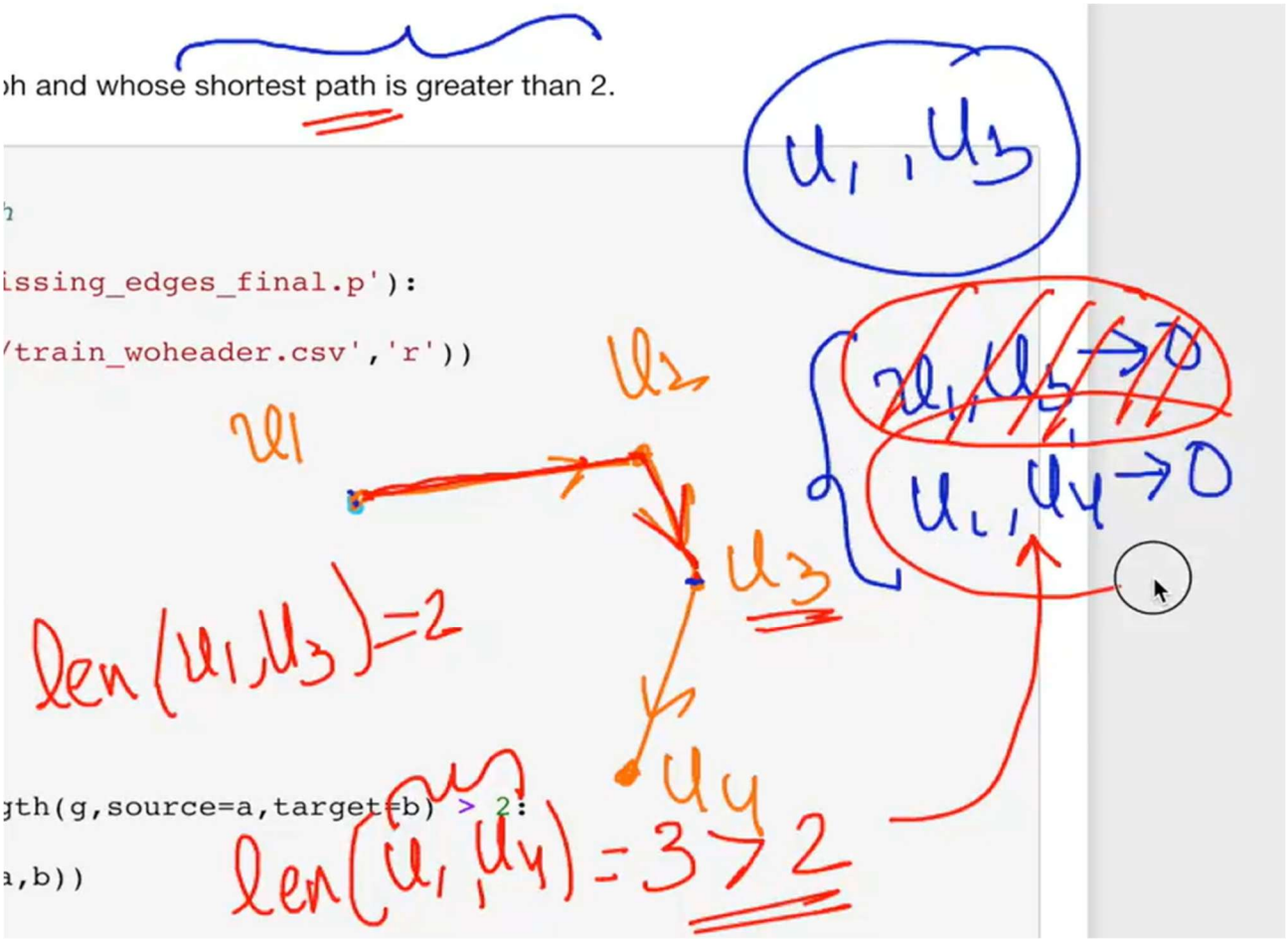
Maltab there are 3 lakh people that only follow or has follower 1.
Max of no of followers + following is 1579
1 persons having maximum no of followers + following.
No of persons having followers + following less than 10 are 1320326. This are our actual target people of the problem to recommend friends.

- **0-0.2M users**: Have very low total activity (~1-2 connections)
- **0.2-0.8M users**: Moderate activity (~2-4 connections)
- **0.8-1.2M users**: Higher activity (~4-8 connections)
- **1.2-1.4M users**: Very active (~8-12 connections)
- **1.4M+ users**: Super active (12+ connections)

## 10) Making it a supervised learning classification problem

Pls read properly. Its very important.

a) Our train.csv only consist of edges with class 1 that is it only tells the user who follows the other users. It does not tells u about the user who does not follow the other which is nothing but class 0. If we train only with class 1, we won't get any recommendations properly. Therefore we will have to create class 0 by ourself and accordingly make a proper csv file to work further.
Here we will do some other thing that is we don't only make a data like user1 , user 2 , 0. We will have to select only those user who are not interconnected and also who are far away from each other by 2 or more distance. See the image below carefully. If we do U1 and U4 , then we can easily remove U1,U3 also and hence we are first checking whether they are disconnected or not and then their length from the node and accordingly making it as class 0.




We know how important is class balancing. So in the current train.csv we have 94 lakh data of class 1 and hence we will have to create 94 lakh data of class 0.

**Problem Statement & Solution:**

**A) Why We Need Negative Examples (Class 0):**

**Current Issue:**

- Our train.csv only contains **positive connections** (class 1) - people who actually follow each other
- It has **NO negative examples** (class 0) - people who DON'T follow each other
- Without negative examples, our ML model can't learn what makes a "bad" recommendation
- **Result**: Poor recommendation quality because the model only sees "good" connections

**B) Creating Smart Negative Examples:**

**Not Just Random Pairs:** Instead of randomly picking any two users and labeling them as "class 0", we're being strategic:

**Our Strategy:**

1. **Check if users are disconnected** - they shouldn't have any direct connection
2. **Ensure they're far apart** - shortest path between them should be > 2 hops
3. **Why this matters**: We want "hard negatives" that are genuinely unlikely to connect

**C) The Distance Logic (Visual Example):**

- U1 → U3 → U4
- **Good negative pair**: U1 and U4 (distance = 2+ hops)
- **Bad negative pair**: U1 and U3 (distance = 1 hop, they might actually connect!)

**Logic**: If we can easily reach U4 from U1 through U3, then U1 and U4 might become friends through U3's introduction. So they're not a good "negative" example.

**D) Class Balance Requirement:**

- **Positive examples (class 1)**: 9.4 million existing connections
- **Negative examples (class 0)**: Need to create 9.4 million non-connections
- **Goal**: Balanced dataset (50% positive, 50% negative) for optimal ML training

**E) Why This Approach Works:**

- **Quality negatives**: Users who are genuinely unlikely to connect
- **Realistic scenarios**: Mimics real-world recommendation challenges
- **Better model performance**: Learns to distinguish between likely and unlikely connections.

**If distance > 2**: Users are far apart → unlikely to connect → **good negative If distance ≤ 2**: Users are close → might connect through mutual friends → **bad negative If no path exists (exception)**: Users are in different components → **excellent negative**

**Here we are using a python library named pickle to find the missing edges and make its data in a pickle format. Pickle is a Python module that allows you to serialize and deserialize Python objects.**

**The Problem:**

**Your Facebook dataset has people who ARE friends (positive examples). But for machine learning, you also need examples of people who are NOT friends (negative examples).**

**The Challenge:**

**Creating 9.4 million negative examples is very slow because the computer has to:**

1. **Pick 2 random people**
2. **Check if they're already friends**
3. **If not friends, check if they should be friends (are they too close in the network?)**
4. **If they're far enough apart, add them as a negative example**
5. **Repeat this 9.4 million times!**

**This process could take hours!**

**What Makes Pickle So Fast?**

**1. Binary Storage Format**

**Pickle converts your Python objects into binary format (0s and 1s) which is:**

- **Compact: Takes less space on disk**
- **Fast to read: Computers read binary data much faster than text**
- **Direct memory mapping: Can load directly into RAM**

**# SLOW WAY (Regenerating data):**

**# Computer has to:**

**# 1. Generate random numbers**

**# 2. Check if edge exists**

**# 3. Calculate shortest path**

**# 4. Validate conditions**

**# 5. Add to set**

**# 6. Repeat 9.4 million times!**

**# FAST WAY (Using pickle):**

**# Computer just:**

**# 1. Read binary file from disk**

**# 2. Convert binary to Python object**

**# Done!**

| Data Generation (Hours) | Pickle Loading (Seconds) |
|---|---|
| Random number generation | Direct file reading |
| Dictionary lookups (9.4M times) | Binary to object conversion |
| Network shortest path calculations | Memory allocation |
| Condition checking (9.4M times) | Single operation |
| Set operations (add/check) | Bulk data loading |

11) Training and testing spilts

a) If we had time based data then we could have done time-based splitting wich is the most appropriate one because companies like FB,insta etc have ample amount of data with respect to time and hence they work with time based spiltting which is the most optimal solution. But here since we don't have time constraint , the only way to split the data is randomly that is randomly splitting. Will spilt the data into 80-20 ratio that is 80 percent training and 20 percent testing.

| Data | Description | Size |
|---|---|---|
| **X_train_pos** | Training friendships (80% of positive) | ~7.5M pairs |
| **X_test_pos** | Testing friendships (20% of positive) | ~1.9M pairs |
| **X_train_neg** | Training non-friendships (80% of negative) | ~7.5M pairs |
| **X_test_neg** | Testing non-friendships (20% of negative) | ~1.9M pairs |

**Why Split Positive and Negative Separately?**

- **Graph Building**: You need only positive training data to build the friendship network
- **Feature Engineering**: Graph-based features are calculated using only the training friendships
- **Avoid Data Leakage**: Test friendships shouldn't influence feature creation.

We have csv files for all separately for proper working in future also. Now we will check the data overlapping that is , is the data in the train and test both almost same which iwill be good for model training.

Train Graph Info:

Number of nodes: 1780722

Number of edges: 7550015

Graph type: <class 'networkx.classes.digraph.DiGraph'>

Test Graph Info:

Number of nodes: 1144623

Number of edges: 7550015

Graph type: <class 'networkx.classes.digraph.DiGraph'>

Test Graph Info:

Number of nodes: 1144623

Number of edges: 1887504

Graph type: <class 'networkx.classes.digraph.DiGraph'>

no of people common in train and test -- 1063125

no of people present in train but not present in test -- 717597

no of people present in test but not present in train -- 81498

% of people not there in Train but exist in Test in total Test data are 7.1200735962845405 %

Number of edges: 1887504

Graph type: <class 'networkx.classes.digraph.DiGraph'>

no of people common in train and test -- 1063125

no of people present in train but not present in test -- 717597

no of people present in test but not present in train -- 81498

% of people not there in Train but exist in Test in total Test data are 7.1200735962845405 %

**1. Train-Test Split Worked:**

- **Training**: 7.55M friendships (80%)
- **Testing**: 1.89M friendships (20%)
- **Total**: 9.4M friendships split correctly

**2. Node Overlap Analysis:**

| Category | Count | Explanation |
|---|---|---|
| **Common people** | 1,063,125 | People present in BOTH train and test |
| **Train-only people** | 717,597 | People ONLY in training data |
| **Test-only people** | 81,498 | People ONLY in test data ❄️ |

**3. Cold Start Problem Identified:**

% of people not there in Train but exist in Test = 7.12%

**This means:**

- **7.12% of test users** have **NO training data**
- **81,498 people** in test set are completely **new/unknown**
- Your model will struggle to recommend friends for these users

🏠 Facebook Network Visualization:

Training Users (1,780,722):

├── Common with Test: 1,063,125 ☑ (Easy to predict)

└── Train-only: 717,597 (Not tested)

Test Users (1,144,623):

├── Common with Train: 1,063,125 ☑ (Easy to predict)

└── Test-only: 81,498 ❄️ (Cold start problem!)

Cold Start Impact: 7.12% of test predictions will be difficult!

**Why This Happens:**

1. **Random Split**: When you randomly split friendships, some users end up only in test data
2. **Real-World Scenario**: This mimics new users joining Facebook
3. **Realistic Challenge**: Your model must handle users with no history

**Implications for Your Model:**

**Good News (92.88% of test users):**

- Have training data available
- Can calculate graph-based features
- Model will perform well

**Challenge (7.12% of test users):**

- No training friendships
- Cannot calculate network features (common friends, etc.)
- Will need fallback strategies

Expected Performance:

☑ Known Users (92.88%): High accuracy

❄ Cold Start Users (7.12%): Lower accuracy

📊 Overall: Good but not perfect

This is actually a **realistic and valuable** test setup because it mirrors real-world challenges that Facebook's recommendation system faces with new users! 🎯

The 7.12% cold start rate is reasonable - not too high to make the model useless, but significant enough to be a real challenge to solve.

In order to solve this cold start problem, we will have to come up with content based featyres like age, gender, liking and disliking but in our project we only have graph based feature and hence we cannot do so.

Finally we did merged all the train – pos and train – neg and test-pos and test-neg and will further do feature engineering and modelling on this only.

Data points in train data (15100030, 2)

Data points in test data (3775008, 2)

Shape of traget variable in train (15100030,)

Shape of traget variable in test (3775008,)

- **Selective Loading**: You can load only what you need (features OR targets)
- **Memory Management**: Don't need to load everything at once
- **Faster Processing**: Smaller files load faster

- **Focus on Features**: Work with node pairs without target variable clutter
- **Iterative Development**: Test different feature combinations independently
- **Clean Separation**: Features vs. labels are conceptually different

Batch Processing: Load features in chunks for large-scale feature engineering

Cross-Validation: Easily split features without dragging targets along

Model Comparison: Same features, different target encodings.

12) Now we will do some essential EDA on the final train dataset that is train_after_EDA.csv

1) After referring to the CDF, it is completely understood that our majority work will be with new account people or people with less followers.
2) Low (1-10): 653,519 people (35.1%)
3) • Medium (11-100): 1,200,517 people (64.5%)
4) • High (>100): 8,165 people (0.4%)
5) Our data is non-linear

**Key Evidence for Non-Linearity:**

**1. Extreme Skewness & Kurtosis**

- **Skewness = 8.00**: Highly right-skewed (normal = 0)
- **Kurtosis = 148.98**: Extremely heavy-tailed (normal = 3)

**2. Distribution Shape**

- **Power-law behavior**: Your log-log plots show straight lines
- **Heavy tail**: Few users have massive connections, most have very few
- **Not bell-shaped**: Completely different from normal distribution

**3. Statistical Tests**

- **All KS-tests rejected**: No standard distribution fits well
- **Q-Q plots**: Points deviate significantly from straight line
- **CDF analysis**: Shows exponential-like growth, not linear

**4. Social Network Patterns**

- 50% of users have ≤ 7 connections
- 99% of users have ≤ 40 connections
- Top 1% are "super-connectors" with 100s of connections

**What Non-Linear Means Here:**

- **Relationship between variables isn't straight-line**
- **Small changes can have disproportionate effects**
- **Power-law: Y ∝ X^α (curved relationship)**

- **Exponential decay patterns**

  Linear data would show:
- Normal distribution (bell curve)
- Skewness ≈ 0, Kurtosis ≈ 3
- Straight-line relationships in scatter plots
- Uniform variance across ranges

6) **1) Linear vs Non-linear Data**

   **Non-linear** - Your data shows:

   **Skewness: 8.00** (highly right-skewed)

   **Kurtosis: 148.98** (extremely heavy-tailed)

   Power-law-like distribution typical of social networks

   All standard distributions rejected by KS-tests

   **2) Decreasing Failure Rate (Weibull Shape < 1)**

   **Decreasing failure rate** means:

   **Early high activity, then declining** - Users start very active but engagement drops over time

   **"Bathtub curve" - left side** - High initial connection-making, then stabilizing

   **In social networks**: New users make many connections quickly, then slow down

   **Practical meaning**: Most connection activity happens early in a user's lifecycle This is typical for social platforms where users are most active when they first join, then settle into steady usage patterns.

# 3) FB_FE-2.ipynb

1) We will do different types of featurization that is suitable in graph link prediction and also relates in our project. Will have to take both followers and following(followees) into consideration accurately while doing FE.

2) Jaccard Distance

   **Jaccard Distance** measures **similarity between two sets** by comparing their **overlap**.

   Jaccard Similarity = |Common Elements| / |Total Unique Elements|

   = |A ∩ B| / |A ∪ B|

   Person A follows: [Alice, Bob, Charlie, David]

   Person B follows: [Bob, Charlie, Eve, Frank]

   Common people they follow: [Bob, Charlie] = 2 people

   Total unique people: [Alice, Bob, Charlie, David, Eve, Frank] = 6 people

Jaccard Similarity = 2/6 = 0.33 (33% similarity).

For following, people they follow

- **User A** follows: Sports channels, Tech blogs, Music artists
- **User B** follows: Sports channels, Tech blogs, Gaming channels
- **Common interests**: Sports + Tech → **High similarity** → Likely to be friends

For followers,

- **User A's followers**: Friends from college, work colleagues
- **User B's followers**: Same college friends, work colleagues
- **Common follower base** → They move in **same social circles** → Likely friends
- **Followees Jaccard: "Do they have similar interests?"**
- **Followers Jaccard: "Do they know the same people?"**

**Why This Makes Sense for Friend Recommendation:**

**Followees Similarity:**

- If both users follow similar accounts → Similar interests → More likely to be friends

**Followers Similarity:**

- If both users have common followers → They're in the same social network → More likely to know each other

**The Key Point:**

Your functions compare **source user's network** with **destination user's network** to determine if they should be connected - which is exactly what you want for link prediction! 🎯

The similarity measures help predict: *"Should source connect to destination?"* based on their network overlap.

**Why** Train.csv **Nodes Still Give 0:**

Even though the nodes exist in your graph, they likely:

- Have very few connections
- Don't share common neighbors
- Are from different social clusters

**What You Should Expect:**

In a real Facebook dataset:

- **80-90% of node pairs** will have Jaccard = 0
- **Only 10-20%** will have non-zero similarity
- **High similarity (>0.1)** is rare and indicates strong connection potential

**The Reality:**

Getting mostly 0 values is **completely normal** for social network analysis. The few non-zero values are the interesting ones that indicate potential friendships!

3) Cosine Similarity

Cosine Distance/Similarity Explained:

What is Cosine Similarity?

Cosine similarity measures the cosine of the angle between two vectors. It determines how similar two entities are based on their orientation, not magnitude.

Cosine Similarity = $|X \cap Y| / (\sqrt{|X|} \times \sqrt{|Y|})$

**Jaccard asks:** *"What fraction of all unique items do they share?"* **Cosine asks:** *"How proportionally similar are their preferences?"*

**Why Cosine is Useful:**

1. **Handles Size Differences:** Cosine is less affected when users have very different network sizes
2. **Proportional Focus:** Better for finding users with similar **proportional** interests
3. **Geometric Interpretation:** Measures angle between preference vectors

**In Your Facebook Context:**

**For Followees (Cosine):**

- Measures how similar two users' **following patterns** are proportionally
- Less biased against users who follow many vs. few people

**For Followers (Cosine):**

- Measures how similar two users' **audience/follower base** is proportionally
- Good for finding users in similar social circles

**When Cosine > Jaccard:**

- When users have **very different network sizes** but similar proportional interests
- User A follows 10 people, User B follows 100 people, but they share 5 common followees

**When Jaccard > Cosine:**

- When users have **similar network sizes**
- Both users follow roughly the same number of people

**Both metrics complement each other** - Jaccard focuses on absolute overlap, while Cosine focuses on proportional similarity! .

4) **Page Rank**

**What is PageRank?**

**PageRank (PR) is an algorithm used by Google Search to rank websites in their search engine results. PageRank was named after Larry Page, one of the founders of Google. PageRank is a way of measuring the importance of website pages. According to Google:**

**The PageRank of a node is calculated based on the connections it has and the importance of the nodes linking to it.**

- **If a node is linked to by many important nodes, its PageRank will be higher.**
- **The algorithm distributes the importance of a node equally among the nodes it links to, and this process is repeated iteratively until the scores stabilize (converge).**

**PageRank works by counting the number and quality of links to a page to determine a rough estimate of how important the website is. The underlying assumption is that more important websites are likely to receive more links from other websites.The PageRank algorithm outputs a probability distribution used to represent the likelihood that a person randomly clicking on links will arrive at any particular page. PageRank can be calculated for collections of documents of any size**

PageRank is Google's original algorithm that measures the **importance** or **authority** of web pages (or nodes in any network). It was developed by Larry Page and Sergey Brin at Stanford University.

**Core Concept:**

**"A page is important if other important pages link to it"**

- **Not just quantity** of links matters, but **quality**
- A link from a highly-ranked page is worth more than from a low-ranked page
- It's a **recursive** definition - importance depends on importance of others

**In Your Facebook Context:**

**What PageRank Measures:**

- **User Influence/Authority** in the social network
- **"Social Importance"** based on who follows them
- **Quality of followers** matters more than quantity

**High PageRank Users:**

- **Influencers** followed by other influential people
- **Celebrities** with high-quality follower networks
- **Opinion leaders** in specific communities

**Low PageRank Users:**

- **New users** with few connections
- **Followers of few people** or isolated users
- **Users in less connected** parts of the network

**For Friend Recommendation:**

**Why PageRank is Useful:**

1. **Authority Feature**: High PageRank users are more likely to be recommended
2. **Quality Signal**: Users followed by influential people are important
3. **Network Position**: Identifies central/peripheral users
4. **Recommendation Priority**: Suggest connections to high-authority users first

**Feature Engineering:**

- **page_rank_s**: PageRank of source user
- **page_rank_d**: PageRank of destination user
- **page_rank_diff**: Difference in PageRank values
- **page_rank_ratio**: Ratio of PageRank values

**Advantages:**

- ☑ **Link Quality** over quantity
- ☑ **Resistant to spam** (hard to manipulate)
- ☑ **Global importance** measure
- ☑ **Works on any directed graph**

**Limitations:**

- ❌ **Computation intensive** for large graphs
- ❌ **Favors older pages/users** (more time to accumulate links)
- ❌ **Topic-blind** (doesn't consider content relevance)
- ❌ **Static measure** (doesn't change quickly)

**In Social Networks:**

**Interpretation:**

- **High PageRank** = Influential user whose opinions matter
- **Low PageRank** = Regular user with limited network influence
- **PageRank Flow** = How influence spreads through the network

**Friend Recommendation Impact:**

- Recommending connections to high-PageRank users increases user engagement
- Users are more likely to connect with authoritative/popular users
- Helps identify **"bridge users"** who connect different communities

PageRank essentially gives you a **"social credit score"** that measures how well-connected and influential a user is in the network! 🎯

Outputs we got for our dataset.

**Single Line Summary:** The PageRank values are extremely small (all in scientific notation) because they represent probability distributions across ~1.5 million nodes, with the least influential user having PageRank of 1.66e-07, most influential at 2.71e-05, and average influence at 5.62e-07.

**Key Points:**

- 📊 **Small values are normal** - PageRank values sum to 1 across all nodes
- 🎯 **Max/Min ratio ≈ 163** - Shows moderate influence inequality

- 📈 **Mean = 5.62e-07** - Used for imputing missing nodes. Very beneficial for cold start problem that points or nodes those are only in testing not in training.
- 🔢 **Scientific notation** - Handles very small probabilities efficiently

**Interpretation:**

- **High PageRank (2.71e-05)** = Influential users (celebrities, popular accounts)
- **Low PageRank (1.66e-07)** = Regular users with minimal network influence
- **Mean PageRank** = Baseline influence for new/unknown users in recommendations.

5) **Shortest Path**

**If the path between two nodes is short or is less they are likely to be connected and if more then they are not similar. Here in this feature we will only find the shortest path between nodes.**

6) Weekly connected Graphs

Suppose VIT is a very big college. But in that clg I know less number of people personally. But since all the students belong to same clg VIT , there will be a community or cluster of nodes or students who follows each other because they belong to same clg. In these scenario, I can get the recommendation of the students whom I don't know personally but since they belong to the same clg, I can get their recommendation. One of the best example of WCG is Linkdin where I don't know people personally but due to clg they will be recommended to me.

Here the directed graph is treated as undirected graph.

**What are Weakly Connected Components?**

- In a **directed graph**, a **weakly connected component** is a subgraph where all nodes are connected **if the direction of edges is ignored**.
- Essentially, it treats the graph as **undirected** and checks if there is a path between any two nodes in the component.

**Key Characteristics:**

1. **Weak connectivity** ignores edge directions.
2. Nodes in the same WCC are reachable from each other when edge directions are ignored.
3. A graph can have multiple WCCs if some nodes are isolated or disconnected.

**Role in Your Project:**

The function belongs_to_same_wcc(a, b) checks whether two nodes (a and b) belong to the same weakly connected component. This is useful for:

1. **Friend Recommendation:**
    - If two nodes belong to the same WCC, they are more likely to be connected or share mutual connections.
    - Nodes in the same WCC are part of the same "community" or "cluster" in the graph.

2. **Feature Engineering:**
    - The WCC feature (same_comp) is used to determine whether two nodes are in the same component.
    - It helps in predicting the likelihood of a connection between nodes.
3. **Graph Analysis:**
    - Identifying WCCs helps understand the structure of the graph, such as isolated communities or clusters.

**Importance in Your Project:**

- **Community Detection:** Helps identify clusters of users who are likely to interact.
- **Improved Predictions:** Adds a feature that captures the structural relationship between nodes.
- **Graph Mining:** Provides insights into the connectivity and structure of the social network.

From project point of view {code}

**Key Insights:**

1. **Purpose in Project:**
    - The WCC feature (same_comp) helps determine whether two nodes are part of the same community or cluster.
    - This is useful for predicting connections between nodes in the graph.
2. **Efficiency:**
    - Precomputing WCCs (wcc = list(nx.weakly_connected_components(train_graph))) avoids redundant calculations during feature generation.
3. **Graph Structure Analysis:**
    - WCCs provide insights into the connectivity and clustering of nodes in the graph.
4. **Feature Engineering:**
    - The result of belongs_to_same_wcc(a, b) is used as a feature (same_comp) in the training and testing datasets.

---

**Conclusion:**

The WCC algorithm identifies clusters of nodes and checks whether two nodes belong to the same cluster. This feature is critical for understanding relationships between nodes and improving the accuracy of friend recommendations in your project.

## 7) Adamic/Adar Index

Case 1) I and my brother Om follows virat kohli , Virat has a very big followers , so there can be less percentage of chance ki usmai se mujhe koi friend recommendation mile because he has a very big following and does not guarantee the mutual connection.

Case 2) I and one of the person named Kunal munde follows my brother Om and my brother has a less number of follwers , so there is a high chance {mutual connection} that kunal and I be the friend.

The **Adamic/Adar Index** is a graph-based similarity measure used to predict the likelihood of a connection between two nodes in a network. It is particularly useful in link prediction tasks, such as friend recommendations in social networks.

**Formula:**

The Adamic/Adar Index is defined as:

$$A(x,y) = \sum_{u \in N(x) \cap N(y)} \frac{1}{log(|N(u)|)}$$

Where:

- (N(x)): The set of neighbors of node (x).
- (N(y)): The set of neighbors of node (y).
- (N(x) \cap N(y)): The set of common neighbors between nodes (x) and (y).
- (|N(u)|): The degree (number of neighbors) of the common neighbor (u).

---

**How It Works:**

1. **Common Neighbors:** It identifies the common neighbors between two nodes (a) and (b).
2. **Weighting by Degree:** Each common neighbor contributes a weight of (\frac{1}{\log(|N(u)|)}) to the score. This gives higher importance to neighbors with fewer connections (low degree), as they are more likely to indicate a meaningful relationship.

3. **Summation:** The weights of all common neighbors are summed to compute the Adamic/Adar Index.

**Role in Your Project:**

- **Purpose:** The Adamic/Adar Index is used as a feature (adar_index) to predict the likelihood of a link between two nodes.
- **Feature Engineering:** It captures the similarity between nodes based on their shared connections, giving more weight to less-connected neighbors.
- **Friend Recommendation:** Nodes with a higher Adamic/Adar Index are more likely to form a connection, making it a valuable feature for link prediction.

---

**Key Insights:**

1. **Importance of Low-Degree Nodes:**
    - Common neighbors with fewer connections are considered more significant, as they indicate a stronger relationship between the two nodes.
2. **Scalability:**
    - The Adamic/Adar Index is computationally efficient for sparse graphs, making it suitable for large-scale networks like social graphs.
3. **Interpretability:**
    - A higher score indicates a stronger likelihood of a connection between the two nodes.

---

**Conclusion:**

The Adamic/Adar Index is a powerful similarity measure that leverages the structure of the graph to predict links. In your project, it is used as a feature to improve the accuracy of friend recommendations by identifying meaningful relationships between nodes.

8) Is following back

   Simply takes two nodes and check whether they both follow each other or not and if not , then it's a great feature that will help to recommend.

9) Katz Centrality

   Very similar to Page-rank.

- **PageRank** → importance flows **through outlinks**. A node's score depends mainly on **who points to it** (inlinks), but the weight depends on the **outlinks of the source** (since it divides its score). It's more about "votes" passed from important nodes.
- **Katz Centrality** → counts **all inlinks + indirect links (paths of any length)**. Not just direct connections, but also *"a friend of a friend"* effect, with longer paths discounted using a factor α\alphaα.

  👉 So:
- PageRank = **recursive voting via inlinks (and normalized by outlinks)**.
- Katz = **all paths (direct + indirect inlinks), with decay**.

**Katz Centrality in Simple English**

Katz centrality measures the importance of a node in a graph by considering both **direct connections** (immediate neighbors) and **indirect connections** (connections through other nodes).

**How It Works:**

1. A node gets a higher score if it is connected to other important nodes.
2. It gives more weight to **closer connections** and less weight to **distant connections**.
3. A small constant value ((\beta)) is added to ensure every node has some importance, even if it has no connections.

**Key Points:**

- **Direct connections** contribute more to the score.
- **Indirect connections** (via other nodes) contribute less, but they are still considered.
- A damping factor ((\alpha)) controls how much influence distant nodes have.

**In Your Project:**

Katz centrality is used to calculate the influence of each node (user) in the graph. Nodes with higher Katz centrality are more likely to be influential and form connections, making it a useful feature for friend recommendations.

## 10) HITS score

**HITS Algorithm (Hubs and Authorities) Explained**

The **HITS algorithm** (Hyperlink-Induced Topic Search) is a graph-based algorithm that assigns two scores to each node in a directed graph:

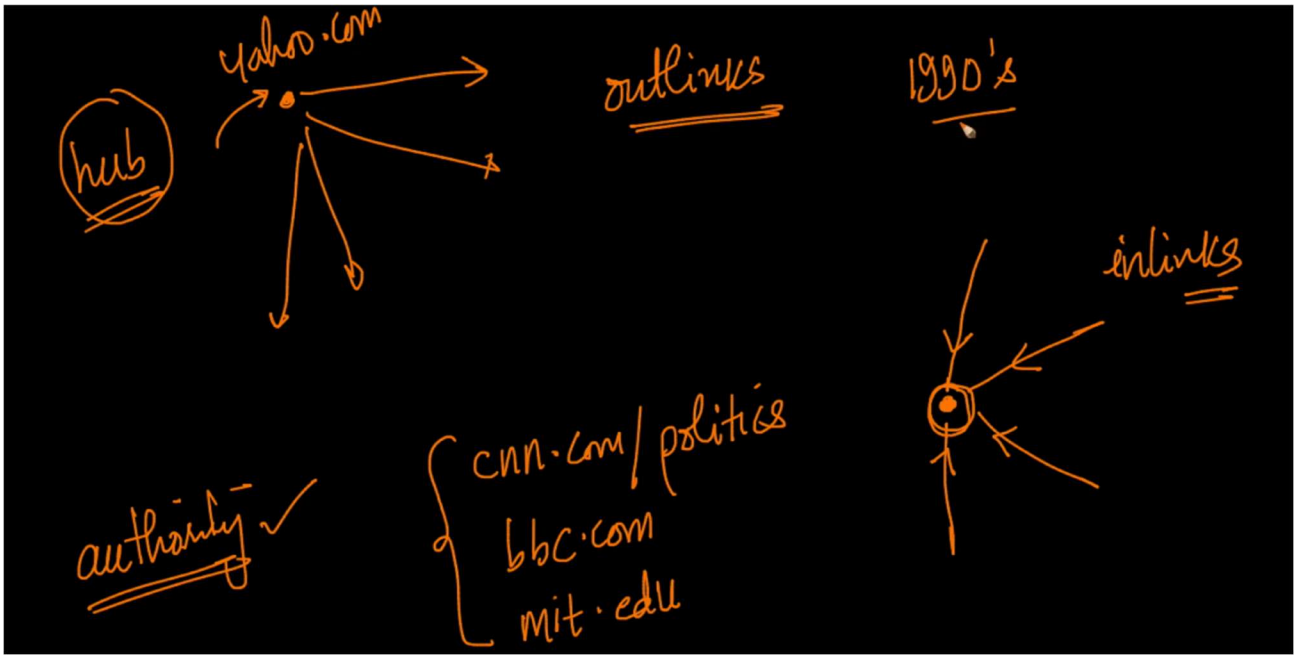
1. **Hub Score**: Measures how good a node is at pointing to other important nodes.
2. **Authority Score**: Measures how important a node is based on the number of good hubs pointing to it.

---

**How It Works:**

1. **Hubs and Authorities Relationship**:
   - A **hub** is a node that points to many **authorities**.
   - An **authority** is a node that is pointed to by many **hubs**.
   - Hubs and authorities reinforce each other: a good hub points to good authorities, and a good authority is pointed to by good hubs.

2. **Iterative Calculation**:
    - Start with an initial score (e.g., 1) for all nodes.
    - Update the **authority score** of a node as the sum of the hub scores of all nodes pointing to it.
    - Update the **hub score** of a node as the sum of the authority scores of all nodes it points to.
    - Normalize the scores after each iteration to prevent them from growing indefinitely.
3. **Convergence**:
    - Repeat the updates until the scores stabilize (i.e., they stop changing significantly).




- **Hubs** are nodes that link to many important nodes (authorities).
- **Authorities** are nodes that are linked to by many important hubs.
- The algorithm is particularly useful for ranking web pages, where hubs are directories and authorities are content-rich pages.

---

**In Your Project:**

- **Hub Scores** (hubs_s, hubs_d) and **Authority Scores** (authorities_s, authorities_d) are calculated for each node.
- These scores are used as features to predict friend recommendations.
- Nodes with high hub or authority scores are more likely to be influential in the network.

**Interpretation in Your Project:**

- **Hub Score**:
    - A user with a high hub score (e.g., User A) is good at pointing to important users (authorities).

- In the context of friend recommendations, this user might be a connector or influencer.

- **Authority Score**:
  - A user with a high authority score (e.g., User B or User C) is considered important because many good hubs point to them.
  - These users are likely to be recommended as friends.

---

**Conclusion:**

The HITS algorithm helps identify:

1. **Influential users** (high hub scores).
2. **Popular users** (high authority scores).

The algorithm also works very simpley by calculating the number of followers and following and then normalizing them and similar doing some other steps also.

Suppose rohan is the person who followes 50 people bbut all the 50 are useless or less popular people bnut at the same time adi foloows 20 people among which 9 are influecer and popular people, the what will be the HUB and authority score here

In this scenario, we can use the **HITS algorithm** to calculate the **Hub** and **Authority** scores for Rohan and Adi. Let's break it down step by step:

---

**Key Points:**

1. **Hub Score**:
   - Measures how good a person is at pointing to important (popular) people.
   - A person who follows many **influential people** will have a higher hub score.
2. **Authority Score**:
   - Measures how important a person is based on how many **good hubs** follow them.
   - A person followed by many **good hubs** will have a higher authority score.

---

**Scenario Setup:**

- **Rohan**:
  - Follows 50 people, but all 50 are **less popular** (low authority scores).
  - Rohan's **Hub Score** will be low because the people he follows are not influential.
- **Adi**:
  - Follows 20 people, and 9 of them are **influencers** (high authority scores).
  - Adi's **Hub Score** will be higher because he follows many influential people.

The algorithm identifies whether Adi follows meaningful (influential) people or not based on the **Authority Scores** of the people Adi follows. Here's how it works step-by-step:

---

**How the Algorithm Identifies Influential People:**

1. **Authority Scores of Nodes:**
   - Each node (person) in the graph has an **Authority Score** that reflects how important or influential they are.
   - A node's **Authority Score** is determined by the **Hub Scores** of the nodes pointing to it (i.e., who follows them).
2. **Hub Scores of Nodes:**
   - Each node also has a **Hub Score** that reflects how good they are at pointing to influential nodes.
   - A node's **Hub Score** is determined by the **Authority Scores** of the nodes it points to (i.e., who they follow).
3. **Adi's Hub Score:**
   - Adi's **Hub Score** is calculated based on the **Authority Scores** of the people Adi follows.
   - If Adi follows people with high **Authority Scores** (influential people), Adi's **Hub Score** will be high.
4. **Iterative Updates:**
   - The algorithm iteratively updates the **Hub Scores** and **Authority Scores** of all nodes in the graph.
   - Over multiple iterations, the scores stabilize, and the algorithm identifies which nodes are influential (high **Authority Scores**) and which nodes are good at pointing to influential nodes (high **Hub Scores**).

## 11) Weights

**Explanation of Weight Features**

Weight features are graph-based metrics that quantify the importance or influence of nodes in a graph. These features are calculated based on the **in-degree** (number of incoming edges) and **out-degree** (number of outgoing edges) of nodes. The weights are designed to capture the connectivity and significance of nodes in the graph.

In order to determine the similarity of nodes, an edge weight value was calculated between nodes. Edge weight decreases as the neighbor count goes up. Intuitively, consider one million people following a celebrity on a social network then chances are most of them never met each other or the celebrity.

On the other hand, if a user has 30 contacts in his/her social network, the chances are higher that many of them know each other.

`credit` - Graph-based Features for Supervised Link Prediction

William Cukierski, Benjamin Hamner, Bo Yang

**Types of Weight Features**

For a directed graph, weights are calculated separately for incoming and outgoing edges:

1. **Weight of Incoming Edges (weight_in)**:
   - Measures the importance of a node based on the number of incoming edges.
   - Nodes with fewer incoming edges have higher weight_in.
2. **Weight of Outgoing Edges (weight_out)**:
   - Measures the importance of a node based on the number of outgoing edges.
   - Nodes with fewer outgoing edges have higher weight_out.
3. **Derived Weight Features**: Additional features can be derived by combining weight_in and weight_out:

weight of incoming edges</li>

<li>weight of outgoing edges</li>

<li>weight of incoming edges + weight of outgoing edges</li>

<li>weight of incoming edges * weight of outgoing edges</li>

<li>2*weight of incoming edges + weight of outgoing edges</li>

<li>weight of incoming edges + 2*weight of outgoing edges

---

**Purpose of Weight Features**

- Weight features help in understanding the **centrality** and **influence** of nodes in the graph.
- They are useful for tasks like **link prediction**, where the goal is to predict the likelihood of a connection between two nodes.
- By combining weight_in and weight_out, the derived features capture the overall connectivity of nodes.

12) SVD features.

**What is SVD?**

SVD is a mathematical technique used to break down a large matrix into smaller, simpler components. Think of it as a way to summarize a big table of numbers (like your graph's adjacency matrix) into smaller pieces that still capture the most important information.

In your project:

- The **adjacency matrix** represents the graph, where rows and columns are nodes, and the values indicate connections between nodes.
- SVD helps reduce the complexity of this matrix while keeping the essential relationships between nodes.

**How SVD Works (Simplified)**

1. **Input: Adjacency Matrix (Adj)**:
    - This is a large matrix where:
        - Rows = Source nodes.
        - Columns = Destination nodes.
        - Values = Whether there is a connection (edge) between nodes.
2. **Decomposition**: SVD breaks the matrix into three smaller matrices:
    - **U (Left Singular Vectors):** Represents the features of the source nodes.
    - **Σ (Singular Values):** Represents the importance of each feature.
    - **V^T (Right Singular Vectors):** Represents the features of the destination nodes.
      Mathematically:
      Adj = U * Σ * V^T
3. **Dimensionality Reduction**:
    - Instead of keeping all the features, you only keep the top k singular values (e.g., k=6 in your code).
    - This reduces the size of the matrices while preserving the most important information.

**Why is SVD Important for Your Project?**

- Your project involves **friend recommendations** using graph mining.
- SVD helps capture the **hidden relationships** between nodes, even if they are not directly connected.
- By adding these features to your dataset, the machine learning model can make better predictions about which nodes (users) are likely to connect.

---

**Summary**

SVD simplifies the graph's adjacency matrix into smaller components, extracting meaningful features for each node. These features are added to your dataset to improve the accuracy of your friend recommendation model. It helps the model understand the graph's structure and relationships more effectively.

**SVD (Singular Value Decomposition)** does not work on the 30 features you already have in your dataset. Instead, SVD operates on the **adjacency matrix** of the graph, which represents the connections between nodes (users) in the graph.

---

**What SVD Actually Does in Your Project**

1. **Input to SVD**:
    - SVD is applied to the **adjacency matrix** of the graph, not the 30 features.
    - The adjacency matrix is a large matrix where:
        - Rows and columns represent nodes (users).
        - Values represent whether there is a connection (edge) between two nodes (e.g., 1 for connected, 0 for not connected).
2. **Output of SVD**:
    - SVD decomposes the adjacency matrix into three smaller matrices: U, Σ, and V^T.

- From these, the top k singular values (e.g., k=6 in your code) are selected to reduce the dimensionality of the graph.

3. **Generated Features**:
   - The rows of the U matrix represent **latent features** for the source nodes.
   - The rows of the V^T matrix represent **latent features** for the destination nodes.
   - These features capture the **hidden relationships** between nodes based on the graph structure.
4. **Adding SVD Features**:
   - The SVD features (from U and V^T) are added as new columns to your dataset (e.g., svd_u_s_1, svd_u_s_2, ..., svd_u_s_6).
   - These features are **independent** of the 30 features you already have.

---

**Key Difference**

- The 30 features (e.g., weights, Adamic/Adar index, etc.) are manually engineered features based on specific graph properties.
- SVD features are **automatically generated latent features** that summarize the graph's structure in a lower-dimensional space.

No, **k** in SVD does not operate on the 30 features you already have in your dataset. Instead, **k** is used to select the **top k singular values** (and their corresponding vectors) from the **adjacency matrix decomposition**. These singular values and vectors represent the most important patterns in the graph's structure.

---

**What Does k Do in SVD?**

1. **Adjacency Matrix**:
   - SVD is applied to the adjacency matrix of the graph, not the 30 features.
   - The adjacency matrix represents the connections between nodes (users).
2. **Decomposition**:
   - SVD decomposes the adjacency matrix into three components: U, Σ, and V^T.
   - Σ contains the singular values, which represent the importance of the patterns in the graph.
3. **Selecting Top k Singular Values**:
   - Instead of keeping all the singular values, you select only the top k singular values (e.g., k=6 in your code).
   - This reduces the dimensionality of the graph while preserving the most important patterns.
4. **Latent Features**:
   - The top k singular values and their corresponding vectors in U and V^T are used to generate **latent features** for the nodes.
   - These features capture **hidden relationships** in the graph.

---

**What Does k=6 Mean in Your Code?**

- By setting k=6, you are telling SVD to keep the **6 most important patterns** in the graph.
- These patterns are used to generate **6 latent features** for each node (e.g., svd_u_s_1, svd_u_s_2, ..., svd_u_s_6).

---

**Does k Select Features from the 30 Features?**

No, **k does not interact with the 30 features** you already have. The 30 features are manually engineered features based on specific graph properties (e.g., weights, Adamic/Adar index, etc.). SVD operates independently on the adjacency matrix to generate **new latent features**.

---

**How SVD and the 30 Features Work Together**

- The 30 features are manually added to the dataset based on specific graph metrics.
- SVD generates **6 additional latent features** that capture hidden relationships in the graph.
- These SVD features are added to the dataset alongside the 30 features, giving the machine learning model more information to work with.

13) Proper evaluating of the code and combing all the features and making a proper train and test { cross-validate } data.

1) First we have randomly selected 1 lakh data from the train.csv which consist of 15100000 data.

2) From the test data we are selecting 25k data for the cross – validation.

3) We will initially compute all our features on these 1 lakh data and 25 k data and then make a proper train data and do modelling and if we get the best results after using this 1 lakh data then we will also check by evaluating the total 15000000 data and 37 lakh test data and compare the results.

4) There are 4 storage_sample_data_ where we will be storing our data step by step in HDF5 format.
HDFStore is a pandas utility for reading and writing data in the HDF5 format.
HDF5 (Hierarchical Data Format) is a file format designed for storing and managing large amounts of data efficiently.

5) In storage_sample_stage1.h5, we have computed the total 9 features and stored - Feature Names: ['source_node', 'destination_node',

'indicator_link', 'num_followers_s', 'num_followers_d', 'num_followees_s', 'num_followees_d', 'inter_followers', 'inter_followees']
Number of Features: 9

6) In storage_sample_stage2.h5, we computed the features like
adar index
is following back
belongs to same weakly connect components
shortest path between source and destination
Feature Names: ['source_node', 'destination_node', 'indicator_link', 'jaccard_followers', 'jaccard_followees', 'cosine_followers', 'cosine_followees', 'num_followers_s', 'num_followees_s', 'num_followees_d', 'inter_followers', 'inter_followees', 'adar_index', 'follows_back', 'same_comp', 'shortest_path']
Number of Features: 16

7) In storage_sample_stage3.h5, we computed the features like

weight of incoming edges

weight of outgoing edges
weight of incoming edges + weight of outgoing edges
weight of incoming edges * weight of outgoing edges
2*weight of incoming edges + weight of outgoing edges
weight of incoming edges + 2*weight of outgoing edges

Page Ranking of source
Page Ranking of dest
katz of source
katz of dest
hubs of sourc
hubs of des
authorities_s of source
authorities_s of dest
Feature Names: ['source_node', 'destination_node', 'indicator_link', 'jaccard_followers', 'jaccard_followees', 'cosine_followers', 'cosine_followees', 'num_followers_s', 'num_followees_s', 'num_followees_d', 'inter_followers', 'inter_followees', 'adar_index', 'follows_back', 'same_comp', 'shortest_path', 'weight_in', 'weight_out', 'weight_f1', 'weight_f2', 'weight_f3', 'weight_f4', 'page_rank_s', 'page_rank_d', 'katz_s', 'katz_d', 'hubs_s', 'hubs_d', 'authorities_s', 'authorities_d']
Number of Features: 30

8) In storage_sample_stage4.h5, we are adding svd feature and this is the final train data on which we will do modelling.
   We have a total of 54 features and 1 lakh rows on which we will do training and 25k for cross validation.
   Index(['source_node', 'destination_node', 'indicator_link',
   'jaccard_followers', 'jaccard_followees', 'cosine_followers',
   'cosine_followees', 'num_followers_s', 'num_followees_s',
   'num_followees_d', 'inter_followers', 'inter_followees', 'adar_index',
   'follows_back', 'same_comp', 'shortest_path', 'weight_in', 'weight_out',
   'weight_f1', 'weight_f2', 'weight_f3', 'weight_f4', 'page_rank_s',
   'page_rank_d', 'katz_s', 'katz_d', 'hubs_s', 'hubs_d', 'authorities_s',
   'authorities_d', 'svd_u_s_1', 'svd_u_s_2', 'svd_u_s_3', 'svd_u_s_4',
   'svd_u_s_5', 'svd_u_s_6', 'svd_u_d_1', 'svd_u_d_2', 'svd_u_d_3',
   'svd_u_d_4', 'svd_u_d_5', 'svd_u_d_6', 'svd_v_s_1', 'svd_v_s_2',
   'svd_v_s_3', 'svd_v_s_4', 'svd_v_s_5', 'svd_v_s_6', 'svd_v_d_1',
   'svd_v_d_2', 'svd_v_d_3', 'svd_v_d_4', 'svd_v_d_5', 'svd_v_d_6'],
   dtype='object')

9) Train Dataset Info:
   Number of rows: 100002
   Number of columns: 54

   Test Dataset Info:
   Number of rows: 50002
   Number of columns: 54

   Feature Names in Train Data:
   ['source_node', 'destination_node', 'indicator_link', 'jaccard_followers',
   'jaccard_followees', 'cosine_followers', 'cosine_followees',
   'num_followers_s', 'num_followees_s', 'num_followees_d',
   'inter_followers', 'inter_followees', 'adar_index', 'follows_back',
   'same_comp', 'shortest_path', 'weight_in', 'weight_out', 'weight_f1',
   'weight_f2', 'weight_f3', 'weight_f4', 'page_rank_s', 'page_rank_d', 'katz_s',
   'katz_d', 'hubs_s', 'hubs_d', 'authorities_s', 'authorities_d', 'svd_u_s_1',
   'svd_u_s_2', 'svd_u_s_3', 'svd_u_s_4', 'svd_u_s_5', 'svd_u_s_6', 'svd_u_d_1',
   'svd_u_d_2', 'svd_u_d_3', 'svd_u_d_4', 'svd_u_d_5', 'svd_u_d_6', 'svd_v_s_1',
   'svd_v_s_2', 'svd_v_s_3', 'svd_v_s_4', 'svd_v_s_5', 'svd_v_s_6', 'svd_v_d_1',
   'svd_v_d_2', 'svd_v_d_3', 'svd_v_d_4', 'svd_v_d_5', 'svd_v_d_6']

Number of Features in Train Data: 54

Feature Names in Test Data:

['source_node', 'destination_node', 'indicator_link', 'jaccard_followers', 'jaccard_followees', 'cosine_followers', 'cosine_followees', 'num_followers_s', 'num_followees_s', 'num_followees_d', 'inter_followers', 'inter_followees', 'adar_index', 'follows_back', 'same_comp', 'shortest_path', 'weight_in', 'weight_out', 'weight_f1', 'weight_f2', 'weight_f3', 'weight_f4', 'page_rank_s', 'page_rank_d', 'katz_s', 'katz_d', 'hubs_s', 'hubs_d', 'authorities_s', 'authorities_d', 'svd_u_s_1', 'svd_u_s_2', 'svd_u_s_3', 'svd_u_s_4', 'svd_u_s_5', 'svd_u_s_6', 'svd_u_d_1', 'svd_u_d_2', 'svd_u_d_3', 'svd_u_d_4', 'svd_u_d_5', 'svd_u_d_6', 'svd_v_s_1', 'svd_v_s_2', 'svd_v_s_3', 'svd_v_s_4', 'svd_v_s_5', 'svd_v_s_6', 'svd_v_d_1', 'svd_v_d_2', 'svd_v_d_3', 'svd_v_d_4', 'svd_v_d_5', 'svd_v_d_6']

Number of Features in Test Data: 54

# 4) FB_Model-3.ipynb

1) See the above dataset info of the train and test properly. We will begin the modelling from now.
2) First we will remove the source and destination columns from the dataset.
   **Why Remove source_node and destination_node?**

1. **Identifiers, Not Features**:
   - source_node and destination_node are just IDs that represent the nodes in the graph.
   - These IDs do not carry any meaningful numerical or categorical information that the model can use to learn patterns.
2. **Features Already Represent Relationships**:
   - During the **featurization process**, you have already created features that describe the relationship between the source_node and destination_node (e.g., Jaccard similarity, Adamic/Adar index, SVD features, etc.).
   - These features are derived from the graph structure and are much more informative than the raw node IDs.
3. **Prevent Data Leakage**:
   - If the model uses source_node and destination_node directly, it might overfit to the specific IDs in the training data, which would not generalize well to unseen data.
4. **Target Variable (indicator_link)**:
   - The indicator_link column is the **target variable** (whether a link exists between the nodes), so it is stored separately in y_train and y_test.

---

**What Happens After Dropping These Columns?**

- After dropping source_node, destination_node, and indicator_link, the remaining columns in df_final_train and df_final_test are the **features** that describe the relationship between the nodes.
- These features are used to train the machine learning model.

---

**What If You Need source_node and destination_node Later?**

- If you need the source_node and destination_node columns for any post-processing (e.g., interpreting predictions), you can always keep a copy of them before dropping.

  source_node and destination_node are removed because they are not meaningful features for the model.
  The featurization process has already transformed the graph structure into useful numerical features that the model can learn from.
  If needed, you can keep a copy of these columns for later use.

Now its very imp phase that is modelling. Since we have non-linear data. We will have to find the best models that works on non-linear data.

For non-linear data, consider machine learning algorithms like **Random Forests**, **Gradient Boosting (XGBoost)**, and **Support Vector Machines (SVM) with an RBF kernel.**

**First lets begin with Random forest.**

## • Random Forest

- Our evaluation matric is F-1 score.
- We tested different depth values and trees first and got to know the best one whose f1 score is close to 1 and does not undergo overfitting or underfitting.
- The F1 score is a measure of a model's accuracy, combining precision (how many predicted positives are correct) and recall (how many actual positives are identified).
- It ranges from 0 to 1:
- Closer to 1: Indicates better model performance.
- Closer to 0: Indicates poor performance.
- Train F1 Score:
- The train F1 score increases slightly as the number of trees (estimators) increases, stabilizing around 0.923.
- This indicates that the model is learning well on the training data.
- Test F1 Score:
- The test F1 score also improves as the number of trees increases, peaking at 250 estimators with a score of 0.9188.

- After 250 estimators, the test F1 score starts to slightly decrease (e.g., 450 estimators has a score of 0.9162).
- This suggests that adding more trees beyond 250 does not significantly improve performance and may lead to slight overfitting.
- Train vs Test Scores:
- The train F1 score is consistently higher than the test F1 score, which is expected because the model is optimized for the training data.
- The gap between train and test scores is small, indicating that the model generalizes well.

**Results Table**

| Estimators | Train F1 Score | Test F1 Score |
|---|---|---|
| 10 | 0.9063 | 0.8746 |
| 50 | 0.9206 | 0.9126 |
| 100 | 0.9239 | 0.9141 |
| 250 | 0.9240 | 0.9188 |
| 450 | 0.9237 | 0.9162 |

- Number of trees to be 250 is the best estimate.
- After proper evalution I got 250 and 35 – depth to be best estimate.

After getting the values of trees and depth. We will still find the more bettwe one using random search.

The reason for performing **RandomizedSearchCV** again, even after evaluating the best number of trees (250) and depth (35), is to **fine-tune other hyperparameters** of the Random Forest model. While you have identified good values for n_estimators and max_depth, RandomizedSearchCV allows you to optimize additional parameters that can further improve the model's performance.

---

**Why RandomizedSearchCV is Useful Here**

1. **Fine-Tuning Other Parameters**:
    - RandomizedSearchCV is optimizing additional hyperparameters like:
        - min_samples_split: Minimum samples required to split a node.
        - min_samples_leaf: Minimum samples required at a leaf node.

- These parameters control how the trees grow and can significantly impact the model's performance.

2. **Validation of Previous Results**:
   - RandomizedSearchCV also tests a range of values for n_estimators and max_depth. This ensures that the previously chosen values (250 trees and depth of 35) are indeed optimal or if slight adjustments can improve the model further.
3. **Exploration of Hyperparameter Space**:
   - RandomizedSearchCV explores combinations of hyperparameters in a randomized manner, which can uncover better configurations that were not explicitly tested during manual evaluation.

---

**What You Should Do**

- If you are confident that **250 trees** and **depth of 35** are optimal, you can fix these values and only tune the remaining hyperparameters (min_samples_split, min_samples_leaf, etc.).
- Alternatively, you can allow RandomizedSearchCV to test a range of values for all hyperparameters, including n_estimators and max_depth, to confirm or refine your previous findings.

---

**Conclusion**

Performing RandomizedSearchCV is not redundant; it complements your manual evaluation by optimizing additional parameters and validating your previous choices. Let me know if you need help adjusting the search space or interpreting the results!

We got a AUC of 0.93 - **Key Components of the ROC Curve**

1. **X-axis (False Positive Rate - FPR)**:
   - The proportion of negative instances incorrectly classified as positive.
   - Formula: FPR = FP / (FP + TN).
2. **Y-axis (True Positive Rate - TPR)**:
   - The proportion of positive instances correctly classified as positive (also known as recall).
   - Formula: TPR = TP / (TP + FN).
3. **Curve**:

- The ROC curve plots the trade-off between the **True Positive Rate (TPR)** and **False Positive Rate (FPR)** at various threshold values.
- A perfect classifier would have a curve that hugs the top-left corner.

4. **Area Under the Curve (AUC)**:
   - The AUC score is a single metric summarizing the performance of the model.
   - **AUC = 0.93**: Indicates that the model performs very well in distinguishing between positive and negative classes.

---

**Interpretation**

- **High AUC (0.93)**:
  - The model has strong predictive power and is effective at distinguishing between positive and negative classes.
  - An AUC closer to 1 indicates better performance.
- **Shape of the Curve**:
  - The curve starts near the origin (0, 0) and quickly rises toward the top-left corner, showing that the model achieves a high TPR with a low FPR.
  - This indicates that the model is good at minimizing false positives while maximizing true positives.

---

**Conclusion**

The ROC curve and AUC score of **0.93** demonstrate that the model is highly effective at classification tasks for the test data. It balances precision and recall well, making it suitable for the given problem.

# We checked execution of XGBOOST model and  but got a train F1 score of 0.90 and test F1 score of 0.99 due to which we got a difference of around 0.9 which is indicating the problem of over fitting so hands we came into the conclusion that random forest is the best model for us here in order to deploy.

Then we deployed all the files to the gitub but since the trained random forest model file as well as the feature test CSV file was big be compressed it and put into the github after all we deployed the project on streamlit.